

Homework 2 - Sorting Applications

Course: CIS 387 - Analysis of Algorithms - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: October 1 at 11:59pm

Zip up the 3 java files and upload to Canvas. **Your name must be at the top of each java file**

Note: You may not use any AI assistance or direct outside help on this assignment.

For 2 extra credit points, answer the following questions (leave a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear, or is there any other feedback you would give?

Your responses will be helpful feedback for refining these assignments in future courses.

Grading Information

The homework will be graded according to this rubric. Note that you are being graded on both correctness and efficiency of your algorithms. I highly recommend you refer to the in class and code examples from lesson 11 (9/23). *If you can't complete everything, it is better to submit partial work than miss the deadline.*

Points	Description
20	Effort: Does the program compile and run? Was a reasonable attempt made?
20	Part 1: Is smallestGapFast implemented correctly in $n \lg n$ time using sorts
20	Part 2: Is equalSetsFast implemented correctly in $n \lg n$ time using sorts
40	Part 3: Is TaskSchedule.java complete and correct? Does it use a priority queue correctly?

Setup

All the code and files needed for this homework are available on the course git repository:

<https://github.com/amininger-mu/CIS387F26Code>

You will see some starter files for parts 1 and 2 (**SmallestGap.java** and **SetEquality.java**), as well as the **DoublingTest** program that can measure the performance of those solutions. In addition, there are test files (txt files) you can use to test each part (you should make additional test files for larger cases!).

1. Smallest Gap

For this part, you are given the following problem to solve:

Given an array of n distinct integers, find the smallest gap between any pair of values

For example, given `[4, 11, 1, 20, 13]` the smallest gap is 2 (11-13).

If we try to solve this problem on an arbitrary array, we can come up with an $O(n^2)$ solution (see `SmallestGap.java`). The `smallestGap` function iterates over all pair of values and finds the smallest difference.

Requirements

Your task is to write a better algorithm to solve this problem by implementing

`int smallestGapFast(int[] arr).`

Your solution should take advantage of sorting (using `Arrays.sort`) to improve to $O(n \lg n)$

To test your solution using numbers from a file (like `gap_test_sm.txt`), do the command:

- `cat gap_test_sm.txt | java SmallestGap.java -f`

(You should run additional tests beyond the simple ones provided)

2. Set Equality

For this part, you are given the following problem to solve:

Given two sets (arrays) of n distinct integers, check if they are equal (contain same elements).

For example, sets `[1, 4, 6]` and `[6, 1, 4]` are equal, but not equal to `[1, 6, 2]`.

If we try to solve this problem without sorting, we can come up with an $O(n^2)$ solution (See `SetEquality.java`). The function `boolean equalSets(int[] a, int[] b)` function will return true if:

- Every element of `a` is contained in `b`
- Every element of `b` is contained in `a`

Requirements

Your task is to write a better algorithm to solve this problem by implementing

`boolean equalSetsFast(int[] a, int[] b).`

Your solution should take advantage of sorting (using `Arrays.sort`) to improve to $O(n \lg n)$

To test your solution using numbers from a file (like `set_test_1.txt`), do the command:

- `cat set_test_1.txt | java SetEquality.java -f`

(You should run additional tests beyond the simple ones provided)

3. Load Balancing

For this part, you will develop an algorithm to schedule a set of n tasks to run on a CPU that has k cores. Each task has a given amount of time that it will take to run.

Your job is to decide which tasks to run on which core. The goal is to minimize the total time needed (the time that the final task completes).

While in general this is an NP-Hard problem (no efficient way to find the optimal solution), we will use the *longest processing time first* heuristic to find a reasonable solution. This means we will schedule tasks in order from longest to shortest using the next available core.

Program Specification

Create a java file called **TaskScheduler.java**.

Inside, implement an inner class called **Core** that has these attributes:

1. The core id (from 1 to k)
2. A list of task times assigned to that core
3. The total time the assigned tasks will take (for that core)

You may want to refer to **lessons/lesson11/Huffman.java** for an example of defining an inner class and using it in a priority queue.

The main function should follow these steps:

1. Read 2 integers from standard in: k and n
2. Read n integers from standard in and put them into an array
3. Sort the array of tasks in *descending* order (longest -> shortest)
4. Create a priority queue and create/add k Core objects (with empty task lists)
5. Do the following for each task (in order from longest to shortest):
 1. Remove the core from the PQ with the *lowest* total time
 2. Assign the task to that core (and update its total)
 3. Add the core back to the PQ
6. Print the results, including the tasks assigned to each core, the time for each core, and the total time needed.

Note: You may assume that the input is valid, you do not need error handling for malformed input.

Example

Here is one example (see **tasks_small.txt**), with 2 cores and 3 tasks

```
2 3
4
10
5
```

Example Program Output:

```
cat tasks_small.txt | java TaskScheduler.java
-----
Tasks assigned to Core 1:
> 10
Total Time: 10

Tasks assigned to Core 2:
> 5 4
Total Time: 9

The time needed to finish all tasks is 10
```